

Comparative Study of Classification Algorithms

Binary Classification: Magic Gamma Telescope

Ilaria Casolino

Engineering in Computer Science and Artificial Intelligence

June 14, 2026

1 Introduction and Motivation

This report presents a comparative study of supervised machine learning algorithms applied to the binary classification problem of gamma/hadron events in atmospheric Cherenkov telescopes, using the **MAGIC Gamma Telescope** dataset.

When a very energetic gamma ray reaches the Earth’s atmosphere, it produces a cascade of particles (a *shower*). These particles move extremely fast and, while passing through the air, they emit a very faint blue light called **Cherenkov radiation**. This light reaches the telescope and creates the so-called *shower image*, which allows us to reconstruct the properties of the event.

The main goal of analysing these data is to distinguish between two types of events:

- **showers produced by gamma rays** (the signal of interest for astronomers)
- **showers produced by ordinary cosmic rays**, called **hadronic showers** (the background noise)

Gamma rays come from extreme phenomena in the universe, such as black holes or supernovae. **Hadronic showers**, instead, are generated by common cosmic rays and do not carry useful information about the astrophysical source. For this reason, they must be separated from true gamma signals. Distinguishing these two classes of events is essential for understanding the origin of gamma rays.

This project includes the training and comparison of six classification algorithms—Naive Bayes, Logistic Regression, Decision Tree, Random Forest, and Support Vector Machines (with linear and RBF kernels)—to solve the classification task. The goal is to identify which algorithms perform best.

Although this work is based on an astrophysical context, the ability of a model to recognise complex patterns within weak signals may also be relevant in other fields. For example, a model that learns to distinguish signals that are hard to interpret could be used in medical physics, where identifying different types of radiation is fundamental for cancer therapies that rely on particle or gamma-ray beams.

2 Dataset Description

2.1 Dataset Characteristics

The main characteristics of the dataset are summarized in Table 1.

It is important to note that misclassifying a background event as a signal has more serious consequences than the opposite case. Therefore, appropriate evaluation metrics, such as ROC curves, are preferred over simple accuracy.

Characteristic	Value
Number of instances	19,020
Number of features	10 Hillas parameters (continuous)
Target class	Binary (gamma or hadron)
Missing values	None
Task type	Binary classification
Class distribution	Gamma: 12,332 (64.8%) Hadron: 6,688 (35.2%)

Table 1: Summary of the MAGIC Gamma Telescope dataset

2.2 Hillas Parameters

Shower images recorded by the MAGIC telescope typically appear as elongated clusters. To describe them, a principal component analysis (PCA) is applied, defining an ellipse that represents the image. Hillas parameters describe this ellipse in terms of size, orientation, position, and energy distribution. The main features in the dataset are:

- **fLength** (mm): length of the major axis of the ellipse, representing the longitudinal extension of the shower.
- **fWidth** (mm): length of the minor axis, measuring the transverse extension; gamma showers tend to be narrower.
- **fSize** ($\log_{10}$ of the number of photons): logarithm of the sum of all pixel contents, correlated with the primary energy.
- **fConc**: ratio between the sum of the two brightest pixels and **fSize**. Measures light concentration in the image.
- **fConc1**: ratio between the brightest pixel and **fSize**. Provides a more specific measure of light concentration at the brightest point.
- **fAsym** (mm): distance from the brightest pixel to the ellipse center, projected onto the major axis. Quantifies asymmetry along the major axis.
- **fM3Long** (mm) and **fM3Trans** (mm): third moments along the major and minor axes, describing longitudinal and transverse asymmetry.
- **fAlpha** (degrees): angle of the major axis relative to the center, useful to determine the source direction.
- **fDist** (mm): distance of the ellipse center from the origin, representing the angular distance from the assumed source.

2.3 Data Analysis

To explore the dataset structure, feature relationships, and potential outliers, four types of plots were used: *countplots*, *histograms*, *heatmaps*, and *pairplots*. These visualizations provide an overview useful for planning the next steps in *data preprocessing* and *model implementation and training*.

2.3.1 Class Distribution

The analysis shows a moderate class imbalance: 64.8% of events belong to the gamma class, while 35.2% belong to the hadron class. This must be considered during training as it can affect model performance.

Using the `stratify=y` option when splitting the dataset preserves the original class proportions, ensuring that both the training and test sets are representative of both categories. Additionally, setting `class_weight='balanced'` (where supported) adjusts the class weights automatically: the minority class receives a higher weight, penalizing errors more, which helps the model focus on rare examples and improves discrimination between gamma and hadron events.

2.3.2 Feature Histograms

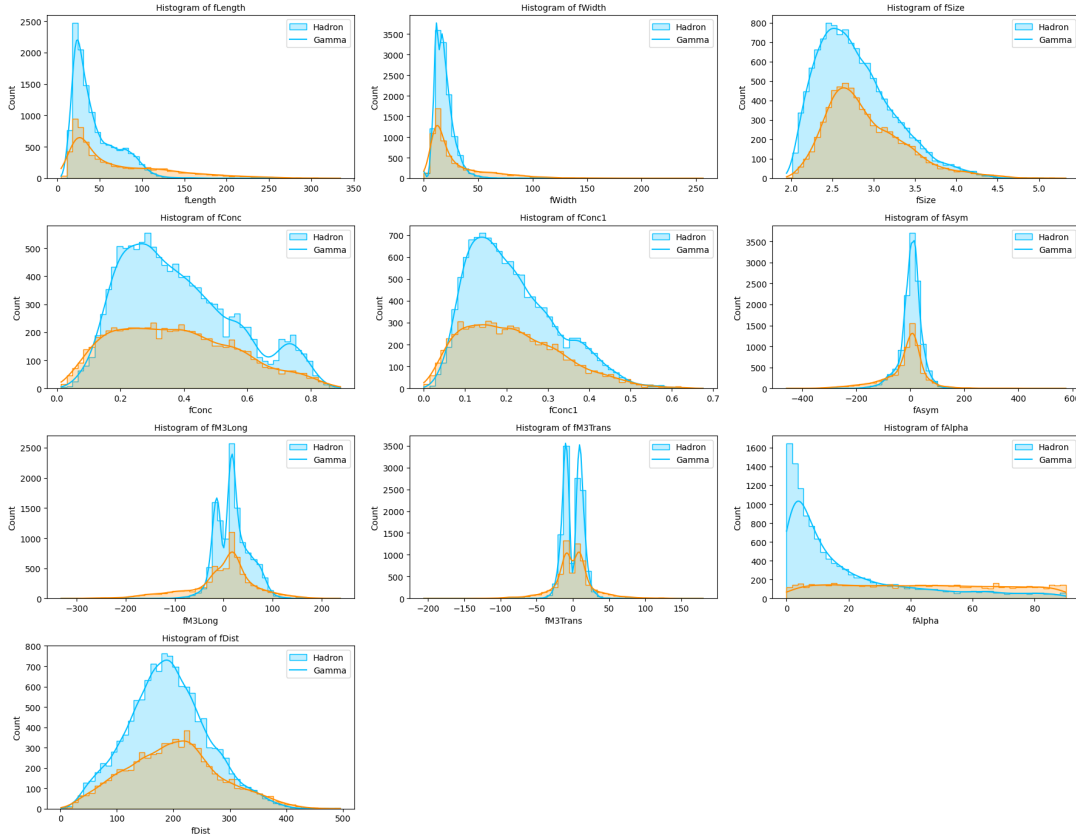


Figure 1: Histograms for gamma (darkorange) and hadron (deepskyblue) events.

Ellipse dimensions

- **fLength** and **fWidth**: Gamma events produce **smaller and more compact ellipses**, concentrated in the lower range of the plots. Hadrons produce **larger and elongated ellipses**, with higher values.

Conclusion: Both features are excellent classifiers, as their distributions overlap minimally.

Energy and light intensity

- **fSize**: Distributions overlap significantly, but gamma events show a clear peak around 2.5-3.0, while hadrons are more spread.

Conclusion: Total energy alone is not enough for classification but is useful when combined with other features.

- **fConc** and **fConc1**: Measure light concentration. Gamma events show **high values** (0.3-0.6), indicating concentrated light, while hadrons have **low values** (0.1-0.3).

Conclusion: Among the most important features, directly reflecting the physical difference between compact gamma showers and diffuse hadronic showers.

Light distribution asymmetry

- **fAsym**: Both classes are concentrated around zero with almost identical distributions.

Conclusion: Not useful alone for classification.

- **fM3Long** and **fM3Trans**: Quantify specific asymmetries. **fM3Trans** shows two distinct peaks for hadrons, indicating complex asymmetries.

Conclusion: Provide additional geometric information but are not primary classifiers.

Orientation and position

- **fAlpha**: One of the most important features. Gamma events are **strongly concentrated near zero degrees**, while hadrons are **almost uniformly distributed** between 0 and 90 degrees. This is because gamma rays come from precise sources, whereas hadrons arrive randomly.

Conclusion: Excellent classifier.

- **fDist**: Distributions peak around 150-200 mm with slightly different tails.

Conclusion: Can provide useful information for classification.

2.3.3 Correlation Matrix

The correlation matrix helps identify redundancies and the most useful features for classification. A near-perfect correlation exists between **fConc** and **fConc1**, suggesting the possible removal of **fConc1**. Other pairs, like **fLength-fWidth**, are significantly correlated but kept, as they capture complementary ellipse shape information. Correlations with the target class confirm that **fConc** is the most predictive, followed by **fWidth** and **fAlpha**. Features like **fAsym** and **fM3Trans** have low predictive power alone.

Removing redundant features can improve linear model stability, but **fConc1** was retained since the total number of features is small and it provides relevant information.

2.3.4 Pairplot

The pairplot shows pairwise relationships between selected features, combining univariate distributions (diagonal) and bivariate scatterplots (off-diagonal), highlighting class separation patterns and outliers.

Scatterplots confirm that combinations like **fAlpha**, **fLength**, and **fWidth** allow good separation between gamma and hadron events; **fSize** combined with geometric features further improves distinction. Some outliers are visible at high values of **fLength**, **fWidth**, and **fDist**, representing rare but real physical phenomena—the tails of the natural distribution. Univariate histograms show overlapping but mostly distinguishable distributions for most selected features.

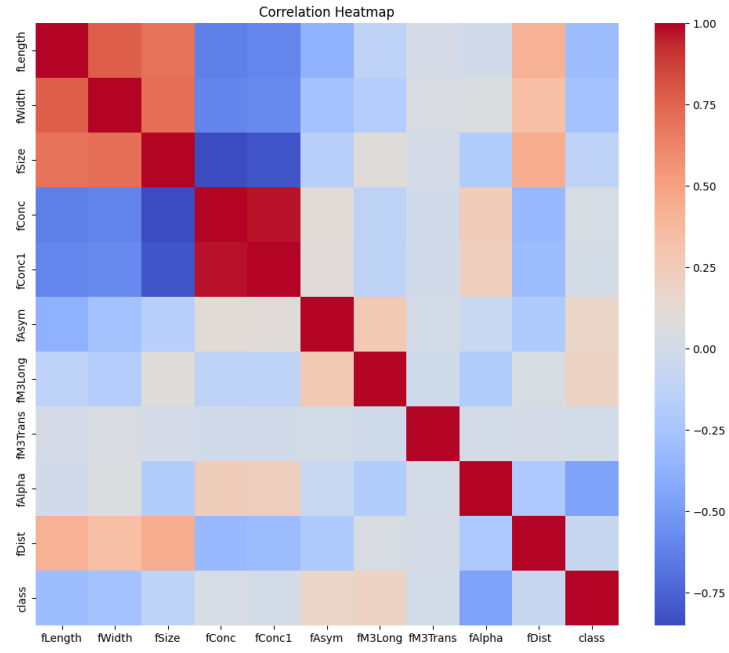


Figure 2: HeatMap.

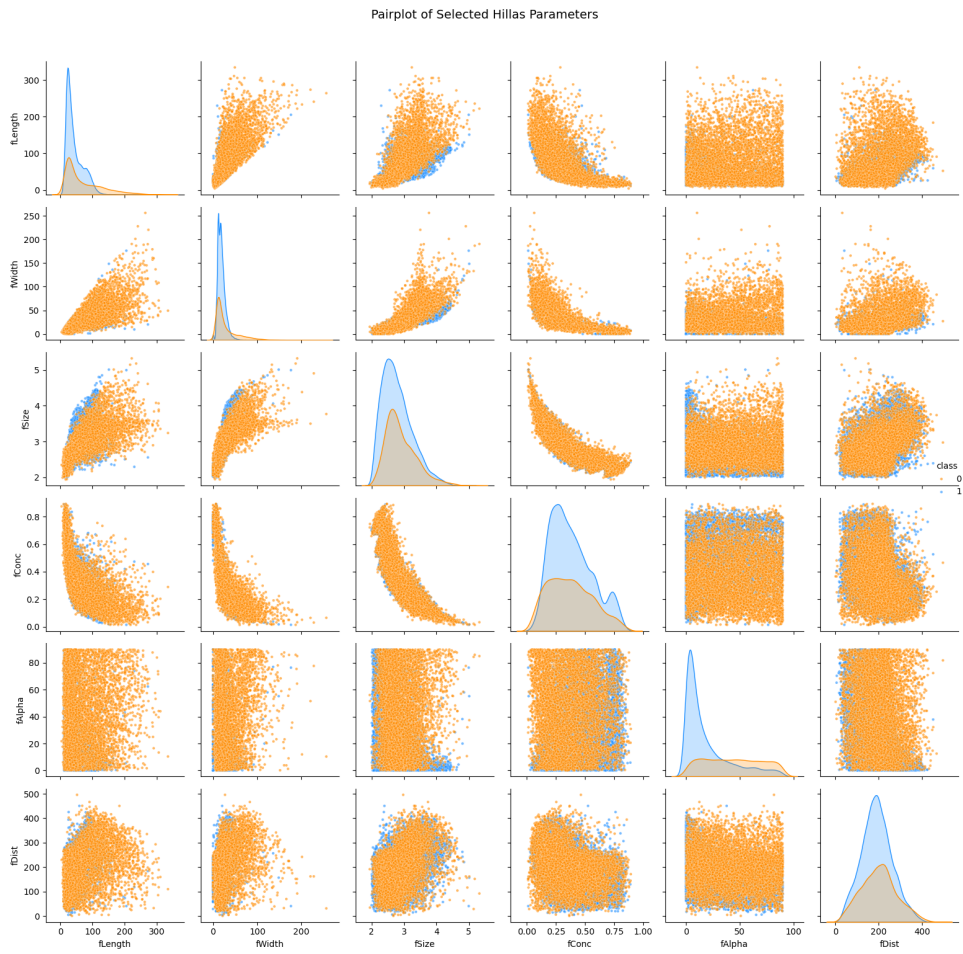


Figure 3: Pairplot of selected Hillas parameters.

3 Methodology and Models

3.1 Data Preprocessing

The **preprocessing** phase is essential to prepare raw data so that they can be correctly used and optimized for machine learning models. The quality of a model strongly depends on the quality of the data, and real-world datasets are *rarely perfect*.

3.1.1 Handling Missing Values

The dataset is exceptionally clean: all 19,020 instances are complete for all 10 features and for the target variable. The absence of missing values and “noisy data” removes the need for imputation techniques (mean, median, KNN imputation), as well as the need to discard samples.

3.1.2 Feature Standardization

The Hillas parameters have very different scales and units:

Feature	Unit / Scale
fLength, fWidth, fDist	Millimetres (about 10–300 mm)
fConc, fConc1	Dimensionless ratios [0, 1]
fAlpha	Degrees [0, 90°]
fSize	Decimal logarithm (typical values 2–5)
fAsym, fM3Long, fM3Trans	Millimetres (negative and positive values)

All features are continuous and heterogeneous. Because the scales differ significantly, some models may be disproportionately influenced by features with larger magnitudes. In particular, SVMs and Logistic Regression, which rely on Euclidean distances or gradients, are sensitive to features on larger scales that may dominate the learning process. For this reason, **Z-score standardization** was applied. This transformation produces features with mean 0 and standard deviation 1.

The `StandardScaler` was **fitted only on the training set**, and the same transformation was then applied to the test set. This separation prevents *data leakage*. When an object such as a **scaler** performs `.fit()`, it learns internal parameters from the data. To avoid contaminating the test set, it must only inherit the scaler information learned from the training set.

Tree-based models (Decision Tree, Random Forest) do not require standardization, since they work with local splitting thresholds. However, all features were standardized for consistency and to make the results easier to compare across models.

3.2 Model Implementation and Training

3.2.1 Dataset Splitting and Cross-Validation

To evaluate model performance, the dataset was initially divided into two separate sets using the `train_test_split` function. The split allocated 75% of the data to the training set (14,265 samples) and the remaining 25% to the test set (4,755 samples).

A separate validation set was not created, since cross-validation already provides an internal split.

All models were trained using **5-fold stratified cross-validation** applied exclusively to the training set. In this procedure, the training data are divided into five folds of equal size while keeping the class distribution balanced. In each iteration, four folds are used for training and one for validation, ensuring that every instance is used exactly once for validation and four times for training. The final metrics are computed as the average over the five iterations.

This method maximizes the use of available data and produces more stable and reliable performance estimates than a single training-validation split. The `cross_validate` function from scikit-learn automates the entire process, returning both training and validation metrics, which are useful for detecting possible overfitting.

For Support Vector Machines, due to their higher computational cost, the cross-validation was reduced to **3 folds** to keep execution times reasonable.

3.2.2 Hyperparameter Tuning

Hyperparameter optimization is essential to obtain the best possible performance from each algorithm. Since optimal configurations depend on the dataset, it is necessary to explore the parameter space. This exploration was performed using `GridSearchCV` and `RandomizedSearchCV`.

GridSearchCV evaluates all possible combinations of the provided hyperparameters. For each combination, the model is trained using cross-validation, and the one with the best performance is selected. `GridSearchCV` was used only for the Naive Bayes model, which has one main hyperparameter (`var_smoothing`) with five candidate values.

RandomizedSearchCV follows a different strategy, randomly sampling a fixed number of configurations from probability distributions defined for each hyperparameter. This method was used for Logistic Regression, Decision Tree, Random Forest, and both SVM variants. The number of iterations depends on the complexity of the model: 20–30 iterations for less demanding models, and 10 for SVMs.

The whole training process for each model was organized inside the `train_and_evaluate_model` function, which handles initialization, hyperparameter search, evaluation, and result storage. First, the base model (for example `LogisticRegression()`) is defined, and the search space is specified using either a dictionary (for `GridSearchCV`) or probability distributions (for `RandomizedSearchCV`). The optimization metric (`scoring='roc_auc'`), the cross-validation strategy, and the number of available cores (`n_jobs=-1`) are also set.

Once the optimal model (`best_estimator_`) is identified, it is applied to the test set using `predict(X_test)` and `predict_proba(X_test)[: ,1]`. These produce, respectively, binary predictions (for the confusion matrix and classification report) and the probability of the positive class, which is required for ROC curve construction. All results, including training times and detailed reports, are stored in global structures (`model_results`, `training_times_list`, `classification_reports`) for later comparison.

3.2.3 Choice of the Optimization Metric: ROC-AUC

The documentation of the dataset guided the decision to use the **ROC-AUC** metric for model optimization.

This metric evaluates the behaviour of the model across *all* possible decision thresholds, not just a fixed one. In Cherenkov telescope analysis, the final threshold is chosen by an expert depending on the scientific goals of the experiment.

Using a metric like the F1-score, which depends on a preset threshold (typically 0.5), would favour models that perform well at that specific threshold, but possibly poorly at the thresholds that are actually used.

Additional reasons come from the different impact of misclassifications, as reported in the *Additional Information* section of the UCI documentation¹. A false positive (hadron classified as gamma) introduces contamination, while a false negative (gamma classified as hadron) only

¹<https://archive.ics.uci.edu/dataset/159/magic+gamma+telescope>

reduces statistical precision. The ROC curve captures this trade-off by showing the True Positive Rate against the False Positive Rate, while metrics like accuracy or F1-score do not distinguish between these two types of error.

In practice, the choice was implemented by setting `scoring='roc_auc'` in all `GridSearchCV` and `RandomizedSearchCV` calls. Other metrics (accuracy, precision, recall, F1-score) are still computed and discussed.

3.3 Models

3.3.1 Naive Bayes (Gaussian)

This model assumes independence between the features, an assumption that is not supported by the dataset, as shown in the previous correlation analysis. For this reason, its performance is expected to be lower than that of the other models.

3.3.2 Logistic Regression

Logistic Regression is a linear model that tries to separate the two classes using a hyperplane in the feature space, assigning each sample a probability of belonging to one class. Its coefficients also help identify which variables have the strongest influence on the model's decisions. However, because it is linear, it cannot capture the non-linear relationships observed in the pairplot, which limits its performance compared to more complex models.

3.3.3 Decision Tree

A Decision Tree is a non-linear model that builds a set of decision rules by progressively splitting the feature space. This allows it to detect complex patterns without requiring data transformations. However, if the growth of the tree is not controlled, the model can easily overfit, which can be observed as a clear gap between training and validation performance in the learning curves.

3.3.4 Random Forest

Random Forest uses many decision trees instead of a single one. Each tree is trained on a slightly different sample of the data and considers only a subset of the features at each split. This double source of randomness reduces overfitting, a common issue in single Decision Trees, and makes the model more stable. Thanks to its structure, Random Forest can capture non-linear relationships between the Higgs parameters. It also provides estimates of feature importance, which are useful to verify whether the model's behaviour is consistent with the findings of the exploratory analysis.

3.3.5 Support Vector Machine - Linear Kernel

The linear SVM looks for the hyperplane that best separates the classes by maximising the margin between them. However, since the decision boundary remains linear, the model cannot capture the more complex relationships in the dataset. In addition, the algorithm has a high computational cost and is usually slower than Logistic Regression.

3.3.6 Support Vector Machine - RBF Kernel

The SVM with an RBF kernel is the most flexible model, because the kernel trick allows it to create very complex and non-linear decision boundaries. This ability also increases the risk of overfitting, especially when `C` and `gamma` are not well balanced. Moreover, training is

considerably slower than for the other models, since computing the kernel is computationally expensive.

4 Results and Analysis

The analysis of the results was carried out by comparing the models both through performance metrics and different visualizations. The goal was to understand which classifiers fit the structure of the dataset best and how their assumptions influenced the final performance.

4.1 Comparative Analysis of Performance

The direct comparison between the models shows clear differences in their classification ability. Random Forest is the best-performing model, with a ROC-AUC of 0.930. It is followed by the SVM with RBF kernel (0.925), and then by the Decision Tree (0.890). Logistic Regression and Linear SVM reach similar values, around 0.845, while Naive Bayes is the weakest model with 0.762.

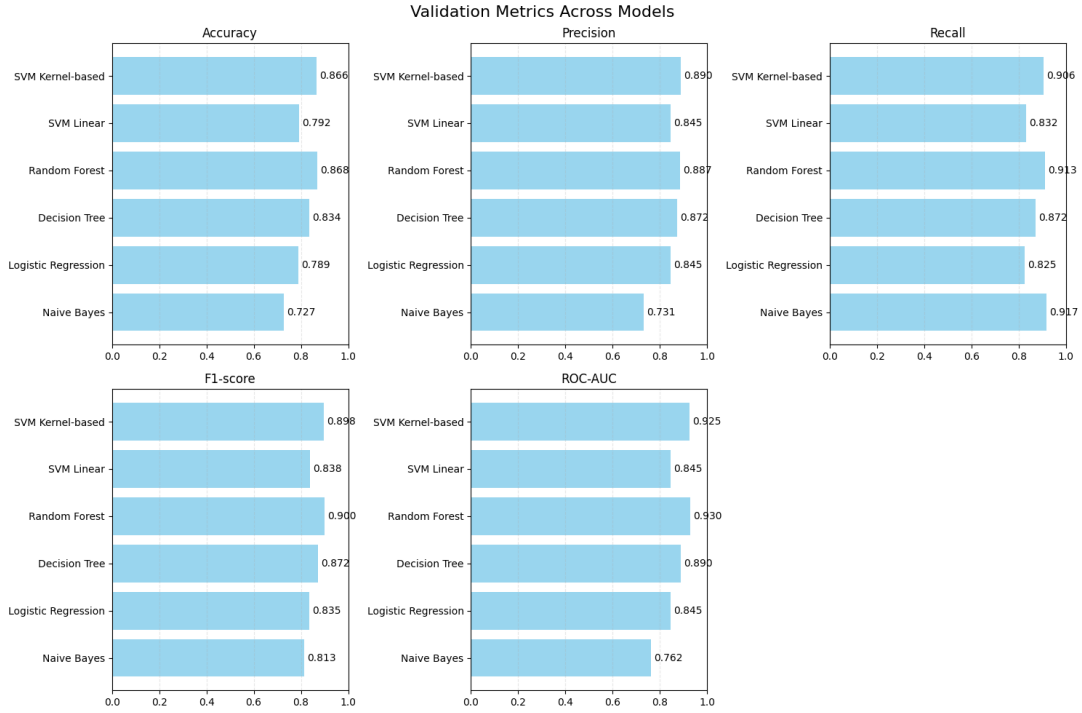


Figure 4: Validation Metrics for All Models

This distribution of results reflects the different levels of complexity each algorithm can model. Random Forest, built as an ensemble of trees, can capture non-linear interactions between the Hilla parameters, which were already visible during the exploratory phase. The presence of strong correlations, such as the one between `fConc` and `fConc1`, benefits models that can manage dependencies between features, while it tends to penalize linear models and Naive Bayes.

The SVM with RBF kernel achieves very high performance (ROC-AUC 0.925, only 0.005 below Random Forest), confirming that a non-linear separation fits the data structure better. However, its computational cost is higher, mainly due to the kernel matrix, which grows quadratically with the number of samples.

Among the linear models, Logistic Regression and Linear SVM show almost identical performance. Their main limitation is the assumption of linearity, which is too restrictive for the gamma/hadron classification task.

4.2 ROC Curves and Discriminative Ability

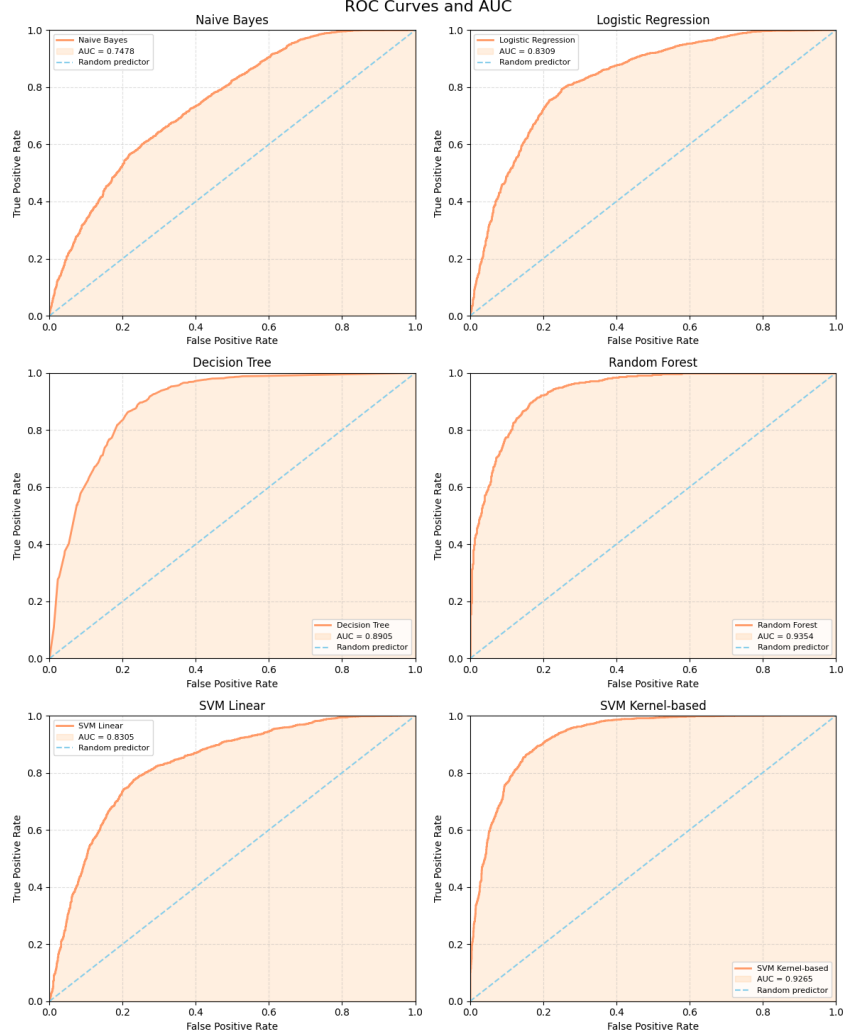


Figure 5: ROC Curves and AUC for All Models

The ROC curves show the trade-off between True Positive Rate (sensitivity) and False Positive Rate for all possible decision thresholds.

Random Forest (AUC = 0.9354) is the model with the curve closest to the ideal top-left corner (TPR = 1, FPR = 0). It increases very quickly even for low FPR values, meaning that the model can reach gamma sensitivity above 90% while keeping false hadron positives below 10%.

SVM with RBF kernel (AUC = 0.9261) has a curve almost identical to that of Random Forest, with small differences in the central area (FPR 0.2–0.4). The AUC difference is very small (0.0093), confirming that it is an excellent alternative.

Decision Tree (AUC = 0.8905) shows a lower curve, especially in the low-FPR region. This

means that to achieve high sensitivity, the single tree must accept more false positives than ensemble methods such as Random Forest.

Linear models (Logistic Regression AUC = 0.8309, Linear SVM AUC = 0.8305) have almost overlapping curves, clearly below the non-linear models. In the critical low-FPR zone, the curve grows more slowly, showing that a linear boundary cannot properly separate the two classes.

Naive Bayes (AUC = 0.7478) is the weakest model. Its curve is close to the diagonal of a random classifier, meaning that it struggles to distinguish gamma from hadron. This confirms that the independence assumption, which is not satisfied in the dataset, strongly limits its performance.

4.3 Model Assumptions and Their Relation to Performance

The behaviour of linear models confirms that the data cannot be separated well by a linear decision boundary. The relationships observed in the pairplot, such as the curved patterns between `fSize` and `fConc`, require more flexible models. As a result, Logistic Regression and Linear SVM do not improve even with more training data and stabilise at relatively low metric values.

Naive Bayes performs significantly worse than linear models (ROC-AUC 0.762 vs 0.845). The classification report highlights its difficulties: the hadron class is classified very poorly, with only 38% recall, meaning that 62% of hadron events are wrongly labelled as gamma (1030 out of 1672). This strong imbalance makes Naive Bayes unsuitable for tasks where sample purity is important.

4.4 Overfitting and Generalization Dynamics

The learning curves, evaluated with ROC-AUC (consistent with the metric used in hyperparameter tuning), make it possible to observe how each model behaves as the training size increases and to detect overfitting or underfitting.

Random Forest shows moderate overfitting. The training score remains very high (about 0.985–0.987), while the validation score stabilises between 0.910 and 0.930. This 5.5–6% gap suggests that the model learns some specific details of the training set. However, after hyperparameter tuning, the gap is smaller than before (7–9%), and the validation curve grows steadily. The final ROC-AUC of 0.930 therefore indicates good generalization.

Decision Tree initially shows strong overfitting: the training score is close to 0.97, while the validation score is around 0.845. As the training size increases, the gap decreases from 12% to about 5%. The training curve slowly decreases and the validation curve increases, showing that the model generalizes better with more data. The regularization parameters (`max_depth=9`, `min_samples_leaf=13`, `min_samples_split=13`) become more effective when more data are available.

SVM with RBF kernel shows very stable behaviour. At the beginning, there is a gap of about 5.5% (training 0.96, validation 0.905), but the curves get closer as the dataset grows. The training score decreases slightly (from 0.96 to 0.943) and the validation score increases (from 0.905 to 0.925), reducing the final gap to only 1.8%. This convergence indicates a good level of regularization. The kernel parameters ($C = 15.83$, $\gamma = \text{'auto'}$) control the model complexity well, achieving one of the smallest gaps while maintaining excellent performance.

Linear models (Logistic Regression and Linear SVM) show clear signs of underfitting. Training and validation curves stay around 0.845, almost parallel, with a small gap (1–2%). Even

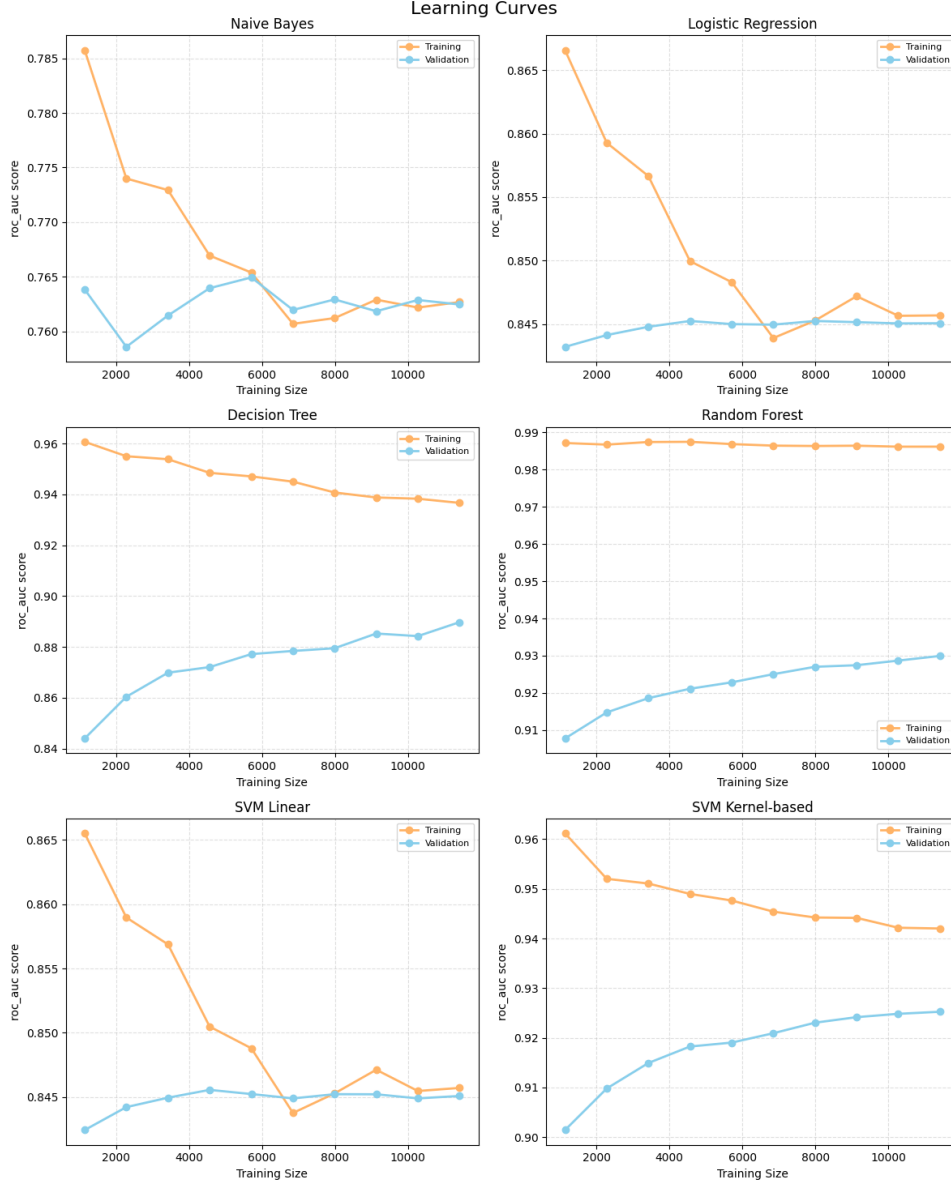


Figure 6: Learning Curves (ROC-AUC)

with more data, there is no improvement, meaning that the limitation is the model capacity. The assumption of a linear boundary is too restrictive for the non-linear relationships in the Hillas features.

Naive Bayes behaves differently: the training score decreases with more data (from 0.785 to 0.762), while the validation score increases slightly (from 0.758 to 0.762). The curves meet because the model becomes worse on the training set rather than better on the validation set. This happens because the independence assumption is violated: as the dataset grows, the probability estimates become less accurate due to feature correlations. The meeting point at 0.76 indicates stable but mediocre performance.

4.5 Feature Importance

The feature importance analysis shows that some Hillas parameters play a crucial role in distinguishing gamma events from hadron events.

fWidth or fSize. This indicates that light concentration is useful, but its information is split between two very similar features.

4.6 Decision Boundaries

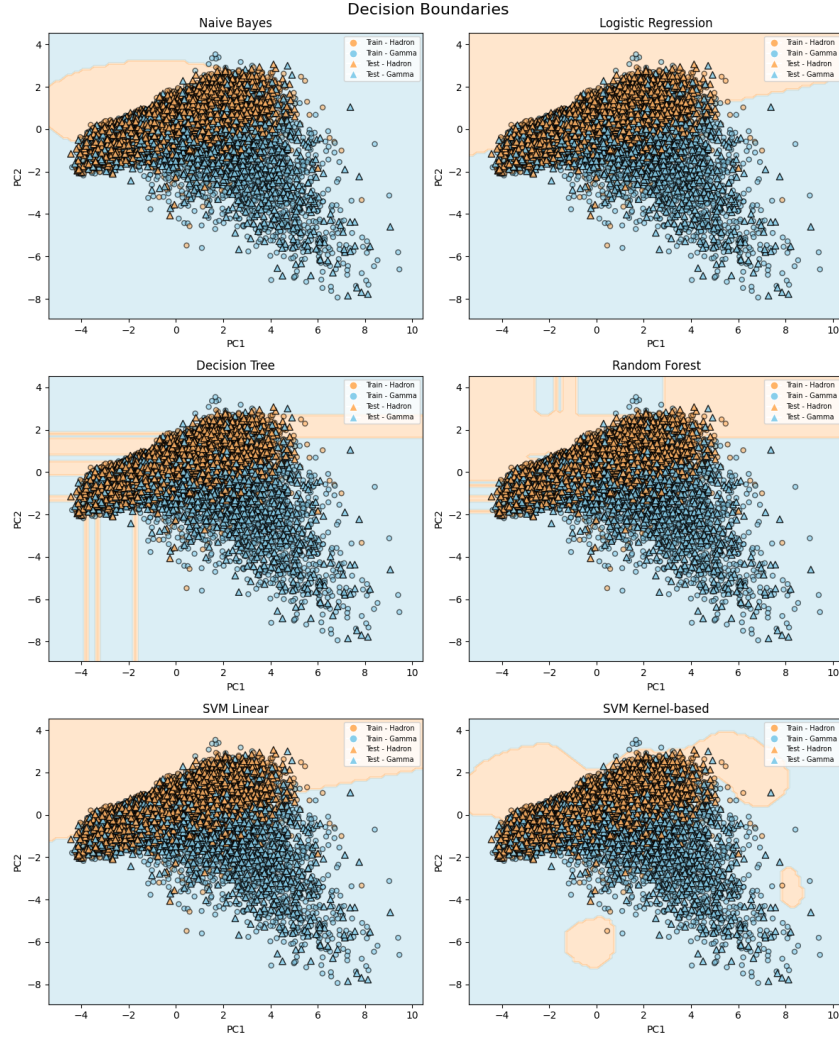


Figure 8: Decision Boundaries in the PCA Space

The plots of the decision boundaries in the two-dimensional PCA space, even if they represent only a projection of the ten original features, allow us to see clear differences between linear and non-linear models.

Linear models, such as Logistic Regression and Linear SVM, produce straight boundaries that roughly separate the two classes. The central region, where the classes overlap, shows that a linear separation is not enough to fully divide them.

The Decision Tree creates a fragmented boundary made of rectangular regions, each linked to a path from the root to a leaf. The presence of small isolated areas, classified differently from the surrounding space, suggests a tendency to overfit.

Random Forest reduces these irregularities by combining 150 trees. The final boundary is smoother but still flexible. The intermediate colour areas represent regions of higher uncertainty, where individual trees disagree in their predictions.

The SVM with a non-linear kernel creates the smoothest and most accurate boundary, closely following the shape of the data. The curved lines wrap around the class clusters in a natural

way.

These observations come from a reduced projection, so some information is lost. However, the fact that the differences between models remain clear even in two dimensions suggests that non-linearity is essential for solving this classification task.

4.7 Confusion Matrices

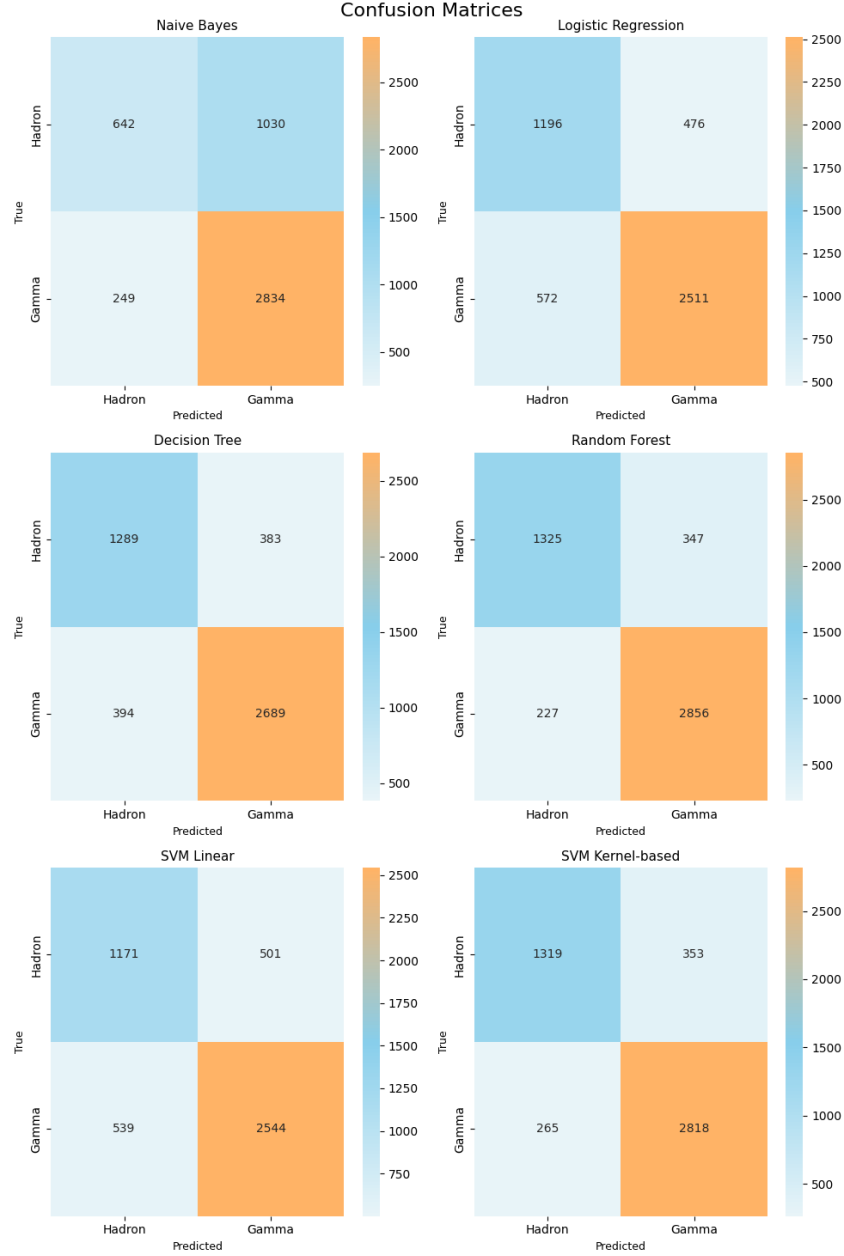


Figure 9: Confusion Matrices on the Test Set

The confusion matrices help us understand the types of errors made by each model.

Random Forest shows a good balance between identifying gamma events and rejecting hadrons. The number of false positives is relatively low (347 out of 1672, equal to 20.7%), and it can be further reduced by adjusting the decision threshold. The model correctly identifies 2856 out of 3083 gamma (recall 0.93) and rejects 1325 out of 1672 hadrons (recall 0.79).

Naive Bayes shows a strong imbalance, with many hadrons incorrectly classified as gamma: 1030 out of 1672 hadrons are misclassified (false positive rate of 62%), and only 642 are correctly recognised. This behaviour reflects the unrealistic independence assumption.

Decision Tree has a more balanced distribution of errors, but the total number of mistakes is much higher than in Random Forest, confirming that the ensemble helps stabilise the decision process.

SVM with non-linear kernel achieves competitive results, with few false positives (353, or 21.1%) and good recall values for both classes (gamma 0.91, hadron 0.79), making it the second-best model after Random Forest.

4.8 Computational Considerations

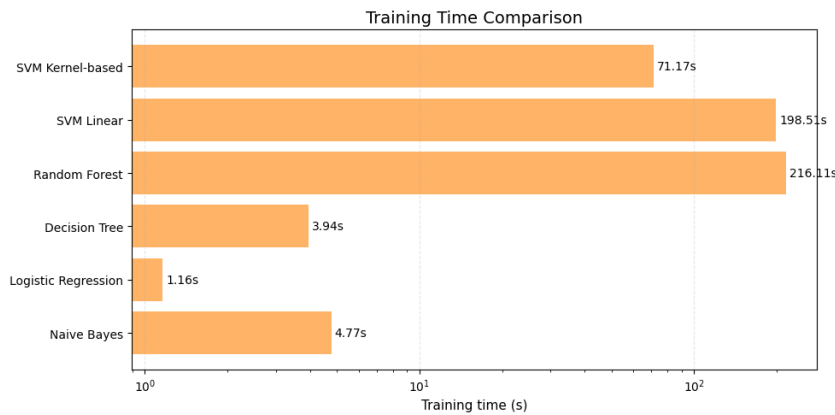


Figure 10: Training Time Comparison

The training time analysis highlights some trade-offs between accuracy and computational cost. **Naive Bayes** is extremely fast (4.77 seconds) and can be useful as a baseline model or in low-resource environments. Logistic Regression is also very quick (1.16 seconds).

Decision Tree trains in a short time (3.94 seconds) and is easy to interpret, even if its performance is lower than more advanced techniques.

Random Forest requires more time (216.11 seconds, about 3.6 minutes), but remains practical for real applications, especially considering its stability and high performance. The increase compared to the earlier version (162 seconds) is due to the 150 estimators and the regularisation parameters used.

Linear SVM shows unexpectedly high cost (198.51 seconds), similar to Random Forest, even if the model is simpler in theory. This is caused by the large size of the training set (14,265 samples) and the quadratic complexity of the optimisation algorithm.

The **non-linear SVM** has a lower training time (71.17 seconds), but this needs context: RandomizedSearchCV tested only 10 configurations for SVM, compared to 30 for Random Forest. The cost per configuration is actually higher for SVM, and the large number of support vectors makes both training and prediction slower. This limits its suitability for large-scale operational scenarios.

5 Conclusions

Considering performance, robustness, interpretability and computational cost, Random Forest emerges as the most balanced solution for the gamma/hadron problem. It achieves the highest

ROC-AUC, and hyperparameter tuning successfully reduced overfitting from an initial gap of 7–9% to 5.5% while keeping strong test performance.

The SVM with RBF kernel shows the best trade-off between overfitting and generalisation, with almost identical training and validation curves (gap 1.8%) and a competitive ROC-AUC (0.925, only 0.005 below Random Forest). It is therefore a strong alternative.

Decision Tree shows overfitting, which improves with more data. Thanks to its interpretability, it still remains a valid third option.

Linear models work well as a comparison baseline or when the data structure fits their assumptions.

Finally, Naive Bayes is not suitable for this problem. Its ROC-AUC of 0.762 (the lowest) and the very poor performance on the hadron class (only 38% recall and 62% false positive rate) make it unreliable in this context.

Overall, the results show that the structural assumptions of the models have a direct influence on their performance.